

1. Explique quais as vantagens da tabela em relação as árvores.
2. Qual as complexidades esperadas para uma tabela hash?
3. Considere uma tabela hash com 13 elementos. Apresente o resultado de inserir na tabela hash utilizando as técnicas abaixo:

18, 41, 22, 44, 59, 32, 31, 73, 85

- a) Utilizando o resto da divisão:

$$h(X) = X \% 13$$

- b) Utilizando o resto da divisão:

$$h(X) = (3X + 11) \% 13$$

- c) Utilizando o resto da divisão:

$$h(X) = (3X + 11) \% 13$$

4. Descreva os conceitos de tratamento de colisões:
 - a) Sondagem linear;
 - b) Sondagem quadrada;
 - c) Duplo Hash.
5. Hash de placas de veículos:
 - a) Um sistema armazena placas de veículos no formato: **ABC1D23**. Descreva um algoritmo que realize a conversão de qualquer placa para um valor entre 1 (placa **AAA1A11**) e 456.976.000 (placa **ZZZ9Z99**).
 - b) Elabore um algoritmo que converte uma placa número como sequência de *char* para um *int*. Nomeie essa função de *hash*. Considere que a função hash deve mapear o mais próximo possível de 25% das posições possíveis das placas.
 - c) Elabore uma nova hash para trabalhar com o padrão antigo de placas (**ABC1234**). Considere que a hash deve mapear aproximadamente 12.5% das placas antigas.
6. Uma tabela hash com m posições possui atualmente fator de carga de 30%. Assume-se que a função hash distribui as chaves uniformemente entre as posições da tabela. Deseja-se inserir uma quantidade adicional de chaves correspondente a 5% da capacidade da tabela. Com o seguinte tratamento de colisão: Se houver colisão, não inserir.
Elabore um algoritmo que simule essas inserções e estime (escolha qualquer função hash de sua conveniência):
 - a) o percentual de novas inserções que sofreram colisão;
 - b) o fator de carga final da tabela previsto.
7. Indique as vantagens e desvantagens de cada tipo de tratamento de colisão.
8. Na **sondagem linear**, os elementos são inseridos na posição seguinte a posição do chave primária hash. Caso também esteja ocupada, é escolhido a posição seguinte e assim por diante.

- Construa uma tabela hash com 100 posições de inteiros disponíveis;
 - Implemente a sondagem linear em dois formatos:
 - sA : O elemento é inserido na primeira posição livre encontrada a partir da posição seguinte, avançando até o final da tabela.
 - sB : O elemento é inserido na primeira posição livre encontrada a partir da posição seguinte, avançando até o final da tabela e reiniciando a busca no início da tabela quando necessário.
 - Implemente um processo de busca para sA e sB.
 - Implemente um processo de remoção para sA e sB.
9. Na **sondagem quadrática**, os elementos são inseridos na posição seguinte a posição do chave primária hash. Caso também esteja ocupada, é escolhido a posição seguinte e assim por diante.
- Construa uma tabela hash com 100 posições de inteiros disponíveis;
 - Implemente a sondagem linear em dois formatos:
 - sA : O elemento é inserido na primeira posição livre encontrada a partir da posição seguinte, avançando até o final da tabela.
 - sB : O elemento é inserido na primeira posição livre encontrada a partir da posição seguinte, avançando até o final da tabela e reiniciando a busca no início da tabela quando necessário.
 - Implemente um processo de busca para sA e sB.
 - Implemente um processo de remoção para sA e sB.
10. Construa uma tabela hash com 100 posições na qual o tratamento de colisões é definido através do processo de listas encadeadas. E depois, implemente:
- O processo de inclusão;
 - O processo de busca;
11. Elabore uma função que gere uma sequência de funções hash no formato:
- $$h(x) = (ax + b) \% K$$
- na qual:
- a: Número primo;
 - b: Número primo;
 - K: Tamanho do filtro de bloom.
12. Elabore uma função para determinar se as funções hashes $h(x)$ determinadas na questão anterior conseguem alcançar todos os elementos da tabela (K elementos).
13. Elabore uma função que determina em quanto tempo seria possível ocupar 50% de todos os elementos da tabela.

14. Determine uma função que, dado um número qualquer e a função hash abaixo, determine qual bit deverá ser selecionado em um filtro de bloom com 16 bytes (vetor de 16 *char*). Considere que os bytes são numerados de 0 a 15 e os bits são numerados de 0(LSB¹) a 7(MSB²)).

$$h(x) = (3x + 7)$$

15. Dado que temos 1024 bytes no filtro de bloom dispersos em um vetor de *char*. Qual a quantidade máxima de elementos que podemos inserir para termos a previsão que obtemos 95% de certeza dos verdadeiros positivos em função da quantidade de funções hash?
16. Projete um filtro que tenha a previsão de inserção de 128 dados utilizando 3 funções hash. Evite que os falsos positivos alcancem 2%.
17. Implemente a função de busca para a questão anterior.
18. Implemente um filtro de bloom com as seguintes especificações:
- Deve guardar os valores primos no conjunto de 2 a N.
 - Deve utilizar 5 funções hash.
 - Deve inserir os primos até a tolerância de 1% para falsos positivos.
19. Implemente a função de busca completa para o filtro de bloom para a questão anterior. Ou seja: Verifique os números primos de 1 até N da seguinte forma:
- Se o elemento não estiver presente, então não é primo.
 - Se o elemento estiver presente, mesmo com uma chance pequena, verifique se de fato é primo.
20. Analise qual algoritmo é mais eficiente: Crivo de erastóstenes tradicional ou Crivo de erastóstenes salvando resultados no filtro de bloom. Realize essa análise para os elementos primos e não primos.
21. Elabore e descreva os algoritmos de inserção e consulta para armazenar todos os números primos no intervalo de 1 a 2.000.000, obtidos por meio do Crivo de Eratóstenes, utilizando um Filtro de Bloom implementado como um vetor de elementos do tipo `unsigned int` (4 bytes).
- Para a implementação, considere que cada posição do vetor representa 32 bits do filtro, de modo que a ativação e a verificação de posições devem ser realizadas por meio de operações bit a bit. Assim, dado um índice i produzido por uma função de dispersão, a posição do vetor correspondente pode ser obtida por $\lfloor i/32 \rfloor$, enquanto o deslocamento do bit dentro do elemento é dado por $i \bmod 32$. Analise se a arquitetura *little-endian* influencia a posição dos bits marcados em um Filtro de Bloom implementado como um vetor de `unsigned int`. Justifique sua resposta.

¹Bit menos significativo

²Bit mais significativo